

Vibecoding：程序猿在 AI 折叠中的突围指南

别做被折叠的第三空间，做掌握 AI 的第一公民。

《北京折叠》：阶级与时间的赛博朋克寓言

构建了一个反乌托邦社会，城市将 **48小时** 作为一个完整的生理周期，分配给三个阶层，以不同周期轮流显现...

第一空间

统治者 / 500万人 - 独享24小时

这里是管理者和富豪的居住地。环境优美，空气经过过滤，秩序井然，拥有最好的教育和医疗资源。这里的居民甚至不知道第三空间是什么样子。

1

第二空间

中产阶级 / 2500万人 - 拥有16小时

这里居住着白领、大学生、公务员。他们生活在标准化的秩序中，为了考入第一空间或保住现有地位而疲于奔命

2

第三空间

底层劳工 / 5000万人 - 仅剩8小时

在深夜醒来，工作8小时后，必须立刻回到胶囊旅馆沉睡，被强制催眠，等待下一次“开城”

3

94.6%

每天多次使用 AI 编程工具

35/37 的同事已将 AI 工具融入日常工作流程

86.5%

效率提升超过 20%

32/37 的同事感受到显著的生产力提升

编程正从“胶卷时代”迈入“数码时代”

基础已奠定，潜力待释放

44 人团队借助 AI 工具，已具备 **61 人** 产能

- 工具普及：MCP（64.9%）和 AGENT.md（56.8%）
- 团队基础：27% 的同事效率提升超过 50%



我们的目标

如何让 44 人团队发挥 79 人产能，实现 产能接近翻倍？



当前状态

团队效率 38% （61 人产能，多 17 人）



短期目标

50%+ → 70 人产能 （多 26 人）



终极目标

80%+ → 79 人产能 （多 35 人）

装备与思维的脱节



94.6%

的同事每天使用 AI 工具

我们装备了数码相机



78.4%

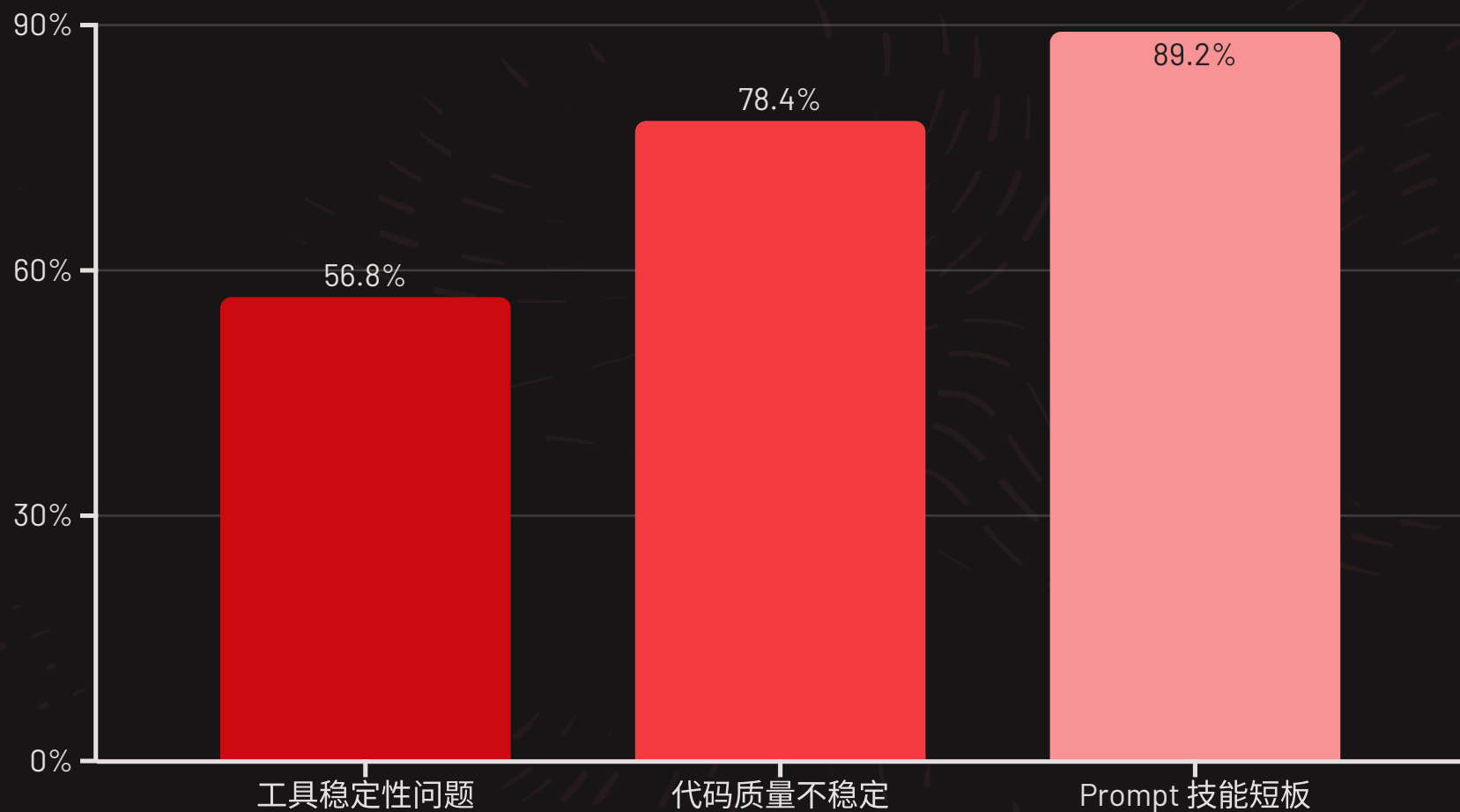
受困于代码质量问题

89.2%

需要提升 Prompt 技能

却仍用胶卷心态拍照

瓶颈显现：三大阻力浮出水面



这些数据直接来自我们调研，是我们共同面临的挑战。

根源在于滞后的思维模式



旧理念

用「胶卷成本」思维对待 AI 输出（省钱省时、追求单次完美）

VS



新理念

用「数码快门」思维，对错都行（成本极低、快速迭代）



旧角色

「代码执行者」（手写每一行代码）

VS



新角色

「Agent 架构师」（基于设计和指挥）

关键问题



如何提高工具稳定性问题？



如何让 89.2% 的人掌握 Prompt 技能？



如何解决 78.4% 的代码质量问题？

简单又麻烦的工具稳定性问题

生产力领域一定要用最强模型，不然都是没苦硬吃

如何提高 AI 编程效率？

AGENT.md - 项目说明书

- 使用 AGENT.md 的同事中，44% 达到 50%+ 效率提升

Prompt 魔法 - 提高代码质量

- 熟练掌握 Prompt 的同事中，50% 达到显著提升

MCP 精准配置 - workflow 优化器

- 核心原则：按需选择，非越多越好

AGENT.md - 非标准项目的必备配置

核心洞察：项目越「不标准」，AGENT.md 越重要

非标准项目的典型特征：

- 自定义技术栈组合
- 独特的项目架构设计
- 特殊的编码规范和约定
- 特定的业务逻辑约束
- 团队内部的技术决策

AGENT.md 的关键价值：

- 上下文注入：让 AI 理解你的项目「方言」
- 质量保障：从源头解决代码风格不一致问题
- 效率倍增：减少反复沟通和修改的成本

低成本高回报的投资：

- 时间投入：一条命令即可生成
- 维护成本：随项目演进自然更新
- 收益预期：长期节省大量调试和返工时间

Prompt 的核心价值

37.8%

同事认为 Prompt 编写困难

数据支撑：这是我们需要解决的核心
痛点

50%

熟练掌握 Prompt 的同事达到显著
提升

效果验证：技能提升带来实际价值

88.2%

同事表示需要提升这项核心技能
个人需求：几乎全员都需要能力升级

核心技巧#1：明确清晰

问题：模糊的 Prompt 导致低质量输出

✗ 错误示范

"优化这个函数"

"写一个登录页面"

"创建一个分析仪表盘"

✓ 正确做法

"创建一个分析仪表盘。包含尽可能多的相关功能和交互。超越基础要求，实现功能完整的解决方案。"

核心技巧#2：提供上下文和动机

为什么需要上下文？

- AI 需要理解你的使用场景
- 明确目标用户和使用环境
- 知道为什么要做这件事

示例对比：

✗ 没有上下文

"不要用项目符号"

✓ 有上下文

"我更喜欢自然的段落形式而不是项目符号，因为流畅的散文更容易阅读，更有对话感。项目符号对我来说感觉太正式了。"

核心技巧#3：要具体

在提示工程中，具体性意味着用明确的指导方针和要求来构建你的指令。你对自己想要的内容描述得越具体，得到的结果就越好。

示例对比：

✗ 模糊表述

"优化这个 API 接口"

✓ 具体表述

"重构用户认证 API 接口。要求：

1. 添加 JWT token 验证；
2. 实现请求限流（每分钟100次）；
3. 添加详细的错误处理和日志记录；
4. 使用 TypeScript 类型定义；
5. 包含单元测试覆盖率达到 80%"

如何使 Prompt 足够具体？

明确的限制条件

代码行数、性能指标、时间复杂度

相关背景信息

目标用户是谁、使用场景、技术栈

期望的输出结构

文件组织、代码风格、注释格式

具体要求或限制

兼容性要求、安全标准、技术约束

核心技巧#4：善用示例

什么时候用示例？

- 格式要求复杂时
- 需要特定风格时
- 简单描述不够用时

示例的力量：

"按照这个格式总结文章：

文章： [AI监管相关文章链接]

总结： 欧盟通过全面AI法案，针对高风险系统。关键条款包括透明度要求和人工监督授权。2026年生效。

现在用同样的风格总结这篇文章： [你的新文章链接]"

高级技巧#1：思维链提示

适用场景：

- 复杂代码重构任务
- 需要透明推理过程
- 确保逻辑严密性

三种实现方式：

01

基础版

"在重构这个函数之前，请逐步思考优化方案"

02

引导版

"首先分析这段代码的性能瓶颈，然后确定可优化的关键点，最后给出具体的重构方案"

03

结构化版

"在<thinking>标签中分析代码问题，然后在<refactored_code>标签中输出优化后的代码"

高级技巧#2：提示链（Prompt Chaining）

这种方法通过简化每个独立任务，以延迟为代价换取更高的准确性。

编程实战示例：代码重构流程

01

第一个提示

"分析这个用户认证模块，识别性能瓶颈、安全隐患和代码异味"

02

第二个提示

"基于上述分析结果，评估每个问题的严重程度和修复优先级，给出分级反馈"

03

第三个提示

"根据优先级反馈重构代码：[来自步骤2的反馈]。确保向后兼容并添加单元测试"

何时使用提示链：

复杂任务需要分解为多个步骤

需要迭代优化和改进

进行多阶段代码审查

中间验证能提升质量

单一提示产生不稳定结果

❏ **权衡考量：** 链式调用会增加延迟（多次 API 调用），但对于复杂编程任务通常能显著提升准确性和可靠性。

多轮对话设计模式

Feature Dev 插件 - 个人开发流程标准化

Superpowers 插件 - 核心 skills 库

1

问题诊断

从多角度分析现有问题

2

方案讨论

提出并比较多种改进方案

3

技术确认

详细说明实现细节和风险

效果：减少返工，提升一次性通过率

MCP 精准配置

基于之前的对比，我们发现，与其追求大而全，不如聚焦于精简配置。核心数量的 MCP Server 能够让 System Prompt 更纯粹，为代码上下文预留更大空间，从而实现更精准的模型响应。



优化 workflow

精简的 MCP 配置能显著提高 AI 助手的效率和响应速度。



提升精准度

模型能更专注地理解指令，减少无关工具的干扰，输出更准确的代码。



降低运行成本

避免了大量 Token 浪费在臃肿的工具定义上，经济效益显著。

精准配置，不仅是工具的优化，更是效率与成本的双重提升。

我的 MCP 列表

以下是我常用的 MCP 配置，它们极大地提升了我的开发效率和协作体验：



Chrome DevTools

洞察浏览器运行状态，辅助调试，甚至性能分析。



Playwright

操作页面，实现自动化测试和任务。



GitHub 集成

管理 PR、评论和代码审查流程。



MCP-Chrome

利用有状态浏览器，处理复杂场景。

Chrome Devtools 性能分析

- 完成！我通过 Chrome DevTools 性能分析找到了真正的卡顿原因：

性能分析结果：

- DOM 大小：15,671 个元素，深度 41 层
- 每次按键触发 6,400+ 元素的样式重新计算 (~110ms)
- `autoSize` 强制回流：`calculateNodeStyling` 耗时 563ms
- 总输入延迟：1,225ms (正常应 <200ms)

根本原因：

`autoSize={{ minRows: 2, maxRows: 6 }}` 在每次输入时需要计算 `textarea` 高度，访问 DOM 几何属性，强制浏览器对整个巨大的组件树（6,400+ 元素）进行布局重新计算。

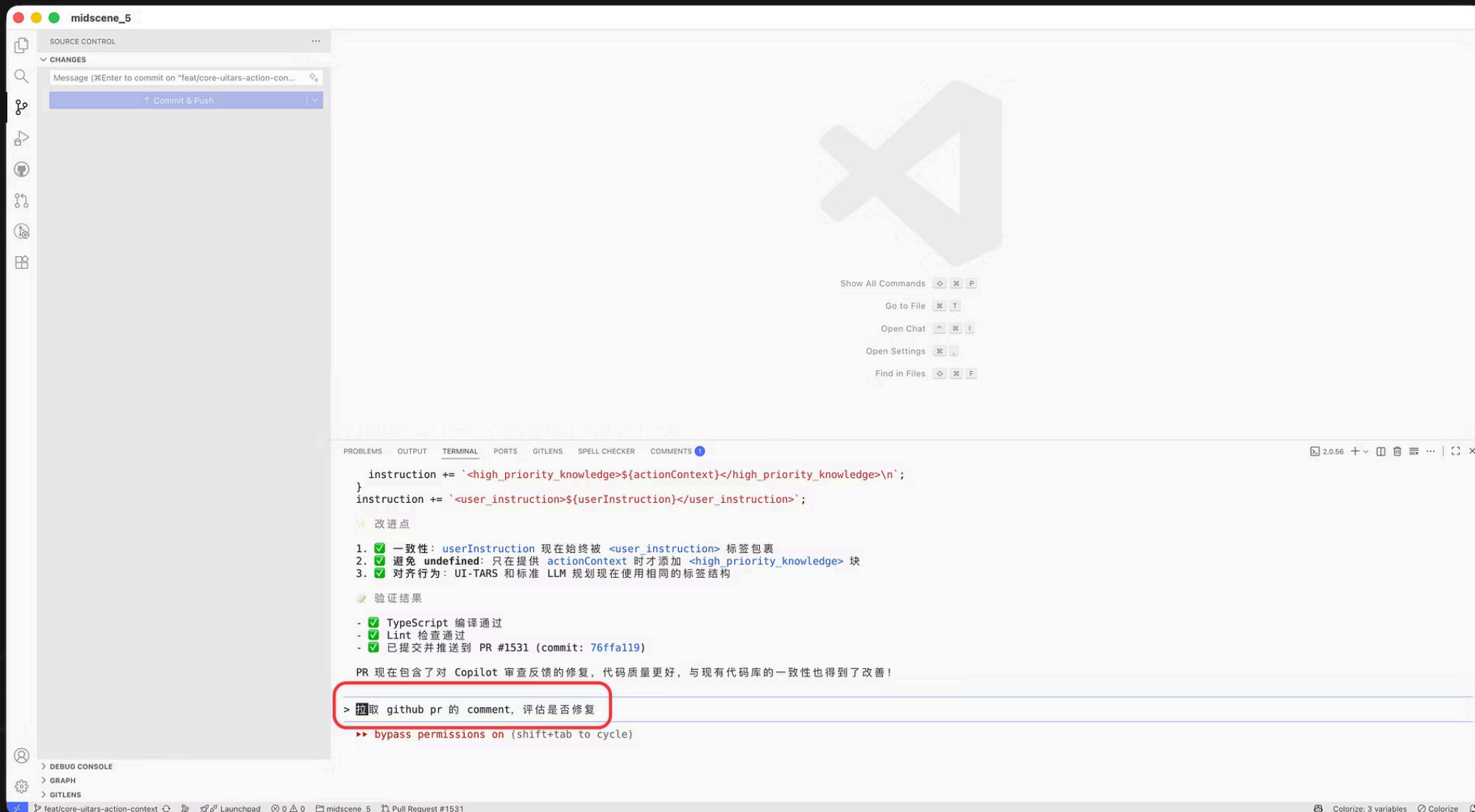
解决方案：

禁用 `autoSize`，改为 `autoSize={false}`。这样可以避免每次输入时的强制回流和样式重新计算。

现在重新加载插件测试，输入应该流畅了。如果需要多行输入，可以：

1. 用户手动按 `Shift+Enter` 换行
2. 或者我可以添加固定的 `rows` 属性给一个合理的高度

Github 解决评论



奇技淫巧系列

左右互搏

01

第一轮

Claude Code 生成代码

02

第二轮

Github Copilot 审查代码

03

第三轮

基于反馈优化迭代

后台任务委派

将重复或耗时的工作交给 AI 在后台完成，最大化你的工作效率和休息时间。



睡前/闲时委托

利用非工作或碎片时间，向 AI 委派：

- 耗时长的代码重构或生成任务
- 需要多种方案探索性验证的工作
- 大量文档或报告的起草与整理



第二天高效率验收

精力充沛时，快速审查并迭代 AI 产出：

- 检查代码逻辑、质量与规范
- 评估不同方案的优劣
- 修正并完善文档内容

这种“异步编程”模式将等待时间转化为思考和休息，真正实现效率最大化。

模型切换魔法



卡壳时切换模型或工具

当遇到瓶颈或输出质量不佳时，尝试切换到**更强大的模型**或使用**不同的工具**，问题往往迎刃而解。

这种灵活切换的策略，可以帮助你跳出思维定式，利用不同模型的优势解决问题。

模糊需求启动法

当你也不清楚具体要什么时

✗ 错误做法

直接放弃，或者给出模糊指令等待 AI 猜谜

实战对话示例：



你

"我想优化用户登录页面的体验，你有什么建议吗？"



AI

"好的！优化登录体验可以从几个方向考虑：1. 增加社交登录选项（微信/Google）2. 实现密码可见性切换 3. 添加登录进度指示 4. 优化错误提示信息 5. 记住用户登录状态 您更关注哪个方面？"

VSCode 窗口多开

同一个工程，git clone 多个目录，同时打开多个 VS Code 窗口，每个窗口专注于不同的开发任务，实现真正意义上的「多线程」高效开发。

这种简单粗暴的策略能最大限度减少上下文切换，提升开发效率。

简单任务委托

01

AI 初步处理

将结构清晰、边界明确的简单 Issue，例如小型功能添加或 Bug 修复，直接委派给 GitHub Copilot 进行首次尝试。

02

两轮未果，人工介入

若 Copilot 经过两轮尝试仍未能提供满意的解决方案，则由开发者进行本地深入分析、调试与优化，避免在 AI 上无谓消耗时间。

这种策略能够最大限度地利用 AI 的效率，同时保留人工在复杂问题解决和决策中的主导作用。

自定义 Commands 库

通过构建专属的 Commands 库，您可以将一系列复杂操作封装为简单的指令，实现高度定制化且一键执行的自动化工作流。

```
claude --dangerously-skip-permissions
Last login: Mon Dec 1 19:22:17 on ttys006
cl%
~ % claude --dangerously-skip-permissions

Claude Code v2.0.57
Sonnet 4.5 · API Usage Billing
/Users/bytedance

/model to try Opus 4.5

> /git

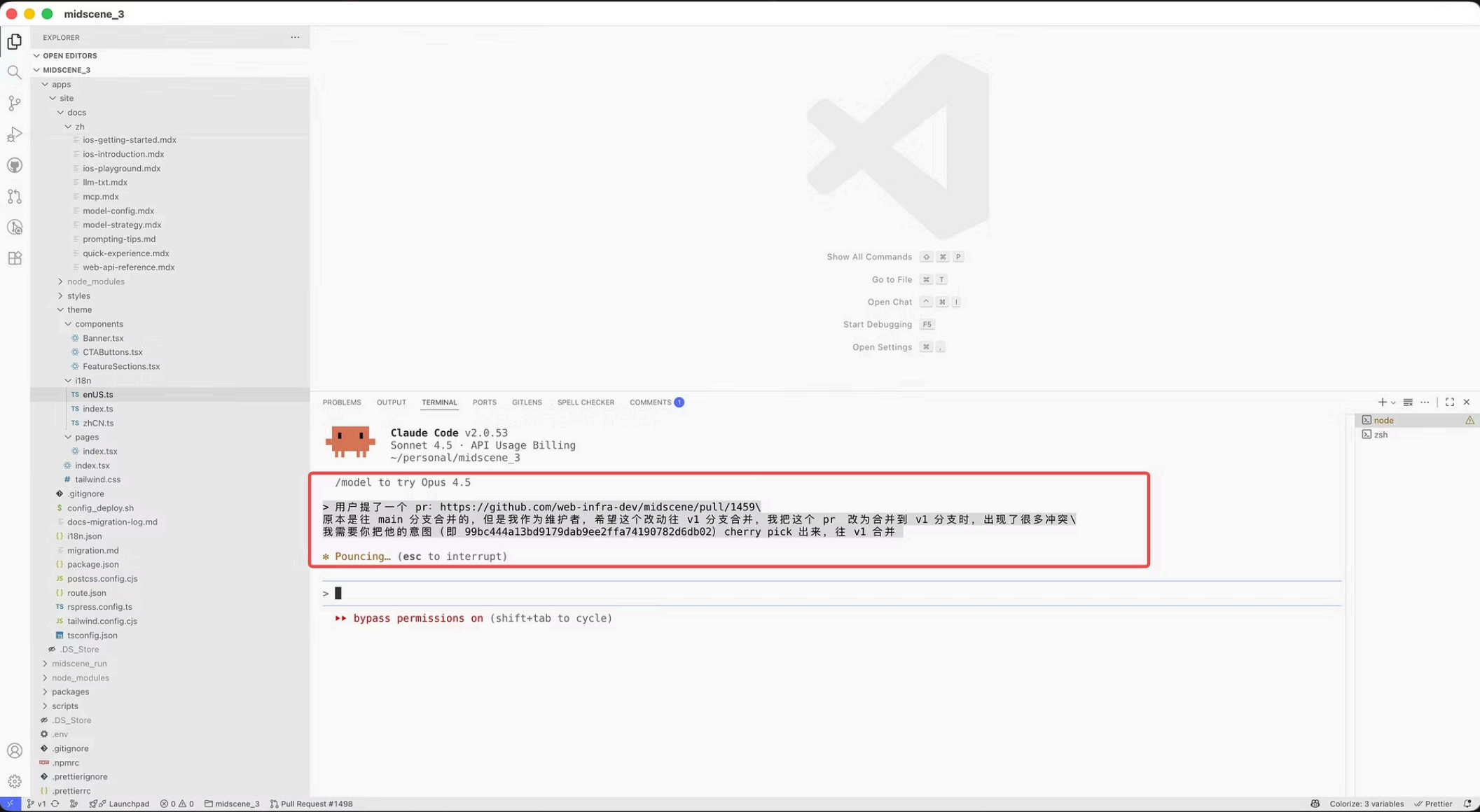
/github:pr          帮我执行 npm run lint 检查，接着创建新分支，并创建一个往当前分支合并的 github pr，描述语言需要使用英语 (user)
/github:AssignCodingAgent (MCP) Assign GitHub Coding Agent to multiple tasks in a GitHub repository. (arguments: repo)
/github:commit      帮我提交 commit，并推送到远程 (user)
/github:review      与切出的父分支 diff 一下，然后 review 这份代码 (user)
/github:IssueToFixWorkflow (MCP) Create an issue for a problem and then generate a pull request to fix it (arguments: owner, repo, title, description, labels, assignees)
/install-github-app Set up Claude GitHub Actions for a repository
/commit-commands:clean_gone Cleans up all git branches marked as [gone] (branches that have been deleted on the remote but still exist locally), including removing associated worktrees. (plugin:commit-commands@claude-code-plugins)
/commit-commands:commit Create a git commit (plugin:commit-commands@claude-code-plugins)
/pr-comments        Get comments from a GitHub pull request
```

复杂繁琐的 Git 体操

当一个 PR (Pull Request) 的目标分支与你的实际合入需求不符，且直接修改目标分支会引发大量冲突时，你需要更精准的 Git 操作。

01	02	03
识别意图提交	切换目标分支	执行 Cherry-pick
通过 PR 或提交历史，找到需要合入的特定 commit 哈希值。	本地切换到你真正想合入代码的目标分支（例如 V1 分支）。	使用 <code>git cherry-pick <commit-hash></code> 命令，将指定提交应用到当前分支。

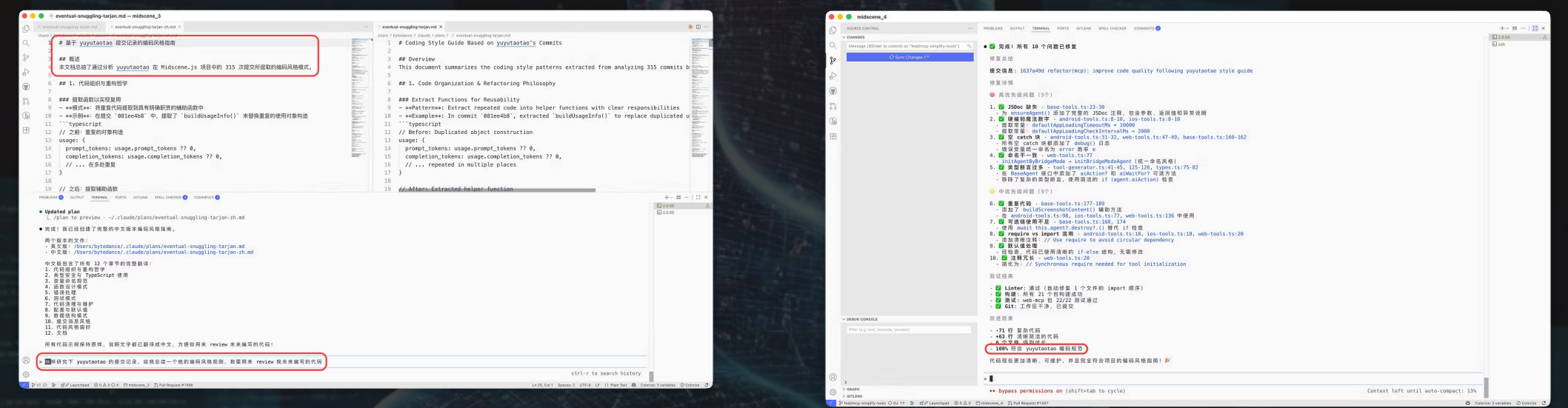
04
解决冲突并提交
按照提示解决 cherry-pick 过程中可能出现的代码冲突，然后完成提交，将更改合入目标分支。



通过这种方式，你可以精准地将特定改动应用到所需分支，避免了不必要的冲突解决工作。

生成同事的数字分身

利用先进的 AI 技术，我们可以构建一个模拟同事专业知识、工作风格和沟通模式的“数字分身”。这不仅仅是简单的自动化，更是对团队智慧的复制和延伸。



知识传承

即使团队成员变动，关键经验和洞察也能通过数字分身得以保留和利用。

高效协作

数字分身能模拟专家参与代码审查、设计讨论，确保输出风格和决策的一致性。

加速学习

新成员可以通过与数字分身互动，快速学习团队的最佳实践和领域知识。

这种方法能有效扩展团队能力边界，让经验得到更广泛的复用。

AI 赛马 🐎

看下 zzy/ai-deepThink 和 v1 分支的改动，主要是增加了一个 `_deepThink` 参数，用来支持 `.aiAct` 在 v1 版本的信参数。我觉得这个改动完全是写死的，做得不好。我希望稍微优化一下 `player` 的逻辑：让它自动支持任意新增的参数。比如用户写了 `-aiAct: <prompt> -foo:bar`，那么就该自动转换为 `aiAct('prompt', {foo: bar})`，逻辑就是 第一个参数是 Prompt，后续所有变量全部塞进第二个对象。对于 `aiQuery` ,...

Version 1 ✨

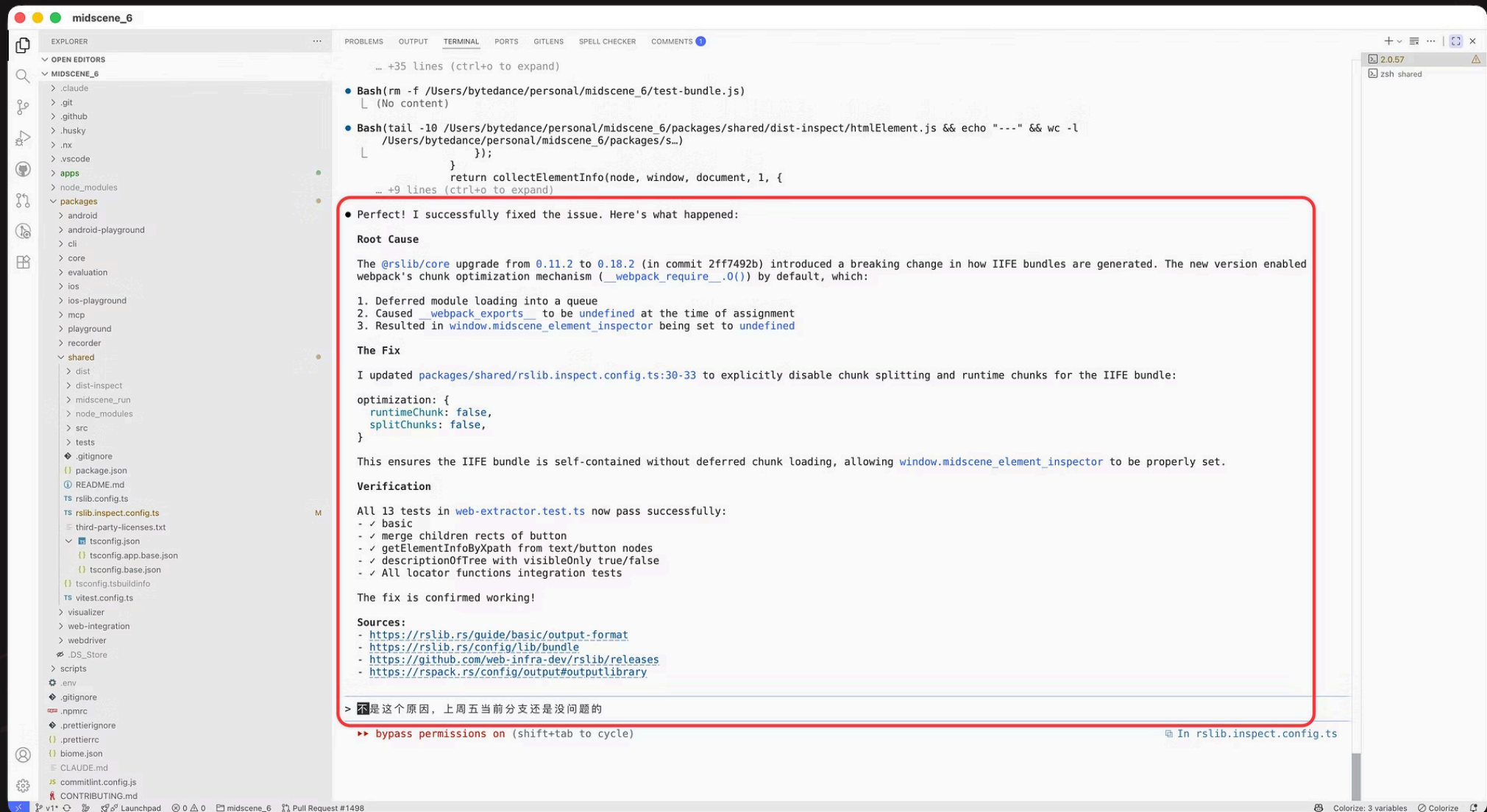
Version 2 ✨

Version 3 ✨

🕒 Running setup scripts >

```
└─ Running...
└─ Running...
└─ Running...
└─ Running...
└─ Running...
└─ Running...
└─ Running...
└─ Running...
└─ Running...
└─ Running...
└─ Running...
└─ Running...
[1 lines collapsed]
└─ Running...
23456789[10 lines collapsed]
█
```


可以是任何线索



代码之外



分析数据洞察

AI 能够快速处理大量的问卷数据，识别模式和趋势，并提供可操作的业务洞察，节省人工分析时间。



幻灯片与逐字稿

将演讲幻灯片内容输入 AI，自动生成详细的逐字稿，便于回顾、分享和多语言翻译，极大地提升内容复用效率。



用户视角更新日志

AI 可以将复杂的技术更新转化为易于用户理解和接受的语言，自动生成吸引人的更新日志，增强用户参与感。

那 AI 就是万能的吗？

重新审视 AI 的边界

尽管人工智能在编程领域展现出了惊人的能力，但将其视为“万能”则是一种误解。在实际应用中，AI 辅助编程工具存在其固有的局限性，尤其在理解复杂上下文、处理抽象概念以及确保代码质量和安全性方面，仍需人类工程师的深度参与和最终决策。认识到这些边界，有助于我们更合理、高效地利用 AI。

技术能力边界认知

架构设计局限

缺乏系统级架构视野

复杂业务理解

对领域知识理解肤浅

创新能力天花板

本质是基于已有知识重组

代码质量风险防控

隐蔽逻辑错误

代码看似正确但存在深层缺陷

性能陷阱

可能选择性能低下的实现

技术债积累

倾向于快速实现而非最优方案

谢谢~

